

Class 7: Machine Learning 1

Mankeerat Rataul

Table of contents

Background	1
K-means clustering	1
Hierarchical Clustering	7
Hands on with Principal Component Analysis (PCA)	9
Analysis of UK food data	9
Data Import	9
Tidy data	10
Exploratory analysis	11

```
library(ggplot2)
```

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionality reduction**.

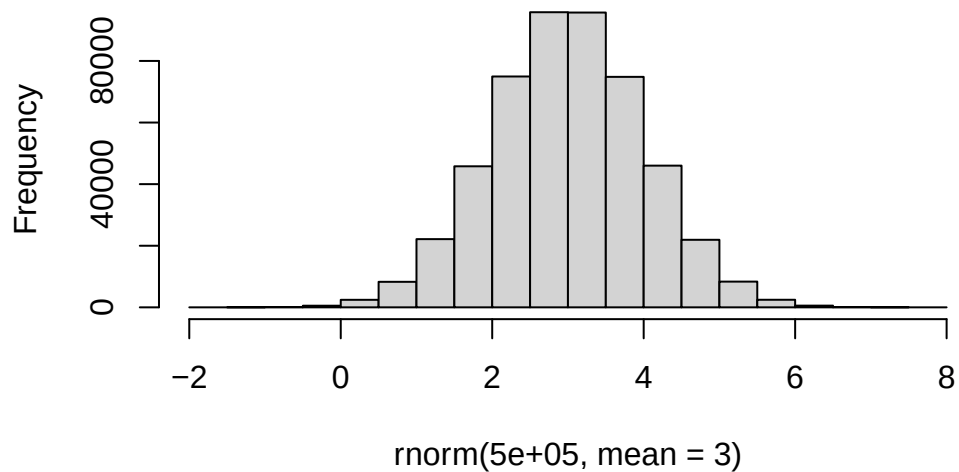
K-means clustering

The main function in “base” R for K-means clustering is called `kmeans()`

Before we go too deep let’s make up some “simple” data that we can cluster and know if we are getting a good answer or not. To do this we can use the `rnorm()` function:

```
hist(rnorm(500000, mean=3))
```

Histogram of rnorm(5e+05, mean = 3)



```
x <- c(rnorm(30, mean=-3), rnorm(30, mean=3))
```

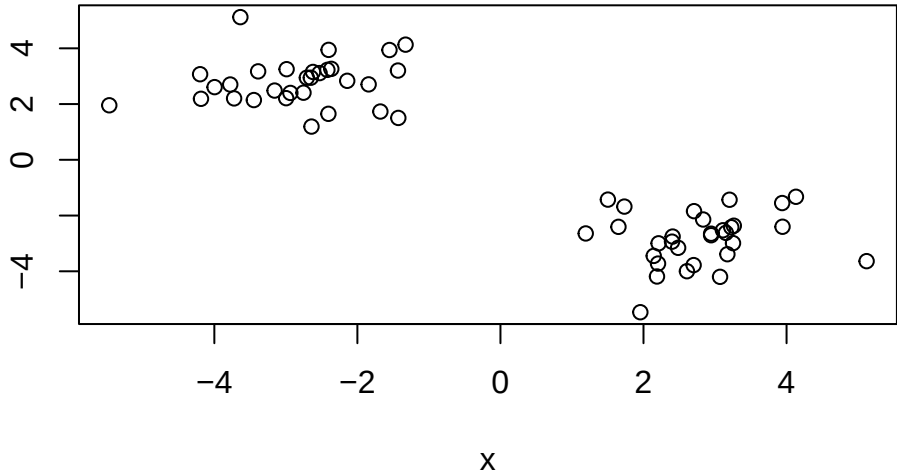
```
z <- cbind(x, rev(x))
```

```
z
```

	x	
[1,]	-2.753750	2.407434
[2,]	-2.936999	2.398632
[3,]	-2.142206	2.834103
[4,]	-2.405441	1.649937
[5,]	-3.723656	2.203051
[6,]	-2.528351	3.110571
[7,]	-2.364699	3.262482
[8,]	-3.777550	2.700772
[9,]	-2.989448	3.251269
[10,]	-2.620849	3.151992
[11,]	-1.678779	1.732394
[12,]	-1.432834	3.203007
[13,]	-3.636184	5.119248
[14,]	-3.387571	3.172853
[15,]	-1.551619	3.938857
[16,]	-5.468818	1.955713
[17,]	-4.198954	3.070397

[18,]	-3.447549	2.143191
[19,]	-2.419346	3.227332
[20,]	-3.996578	2.607253
[21,]	-3.157623	2.484028
[22,]	-2.706525	2.945396
[23,]	-2.402251	3.943623
[24,]	-2.995880	2.210957
[25,]	-2.641859	1.192315
[26,]	-1.427970	1.502267
[27,]	-1.842774	2.706268
[28,]	-1.326456	4.131328
[29,]	-2.651084	2.945386
[30,]	-4.186624	2.188547
[31,]	2.188547	-4.186624
[32,]	2.945386	-2.651084
[33,]	4.131328	-1.326456
[34,]	2.706268	-1.842774
[35,]	1.502267	-1.427970
[36,]	1.192315	-2.641859
[37,]	2.210957	-2.995880
[38,]	3.943623	-2.402251
[39,]	2.945396	-2.706525
[40,]	2.484028	-3.157623
[41,]	2.607253	-3.996578
[42,]	3.227332	-2.419346
[43,]	2.143191	-3.447549
[44,]	3.070397	-4.198954
[45,]	1.955713	-5.468818
[46,]	3.938857	-1.551619
[47,]	3.172853	-3.387571
[48,]	5.119248	-3.636184
[49,]	3.203007	-1.432834
[50,]	1.732394	-1.678779
[51,]	3.151992	-2.620849
[52,]	3.251269	-2.989448
[53,]	2.700772	-3.777550
[54,]	3.262482	-2.364699
[55,]	3.110571	-2.528351
[56,]	2.203051	-3.723656
[57,]	1.649937	-2.405441
[58,]	2.834103	-2.142206
[59,]	2.398632	-2.936999
[60,]	2.407434	-2.753750

```
plot(z)
```



Now we can run `kmeans()` on this input `z` and see what the results look like.

```
km <- kmeans(z, 2)
```

Q. How many points are in each cluster?

```
km$size
```

[1] 30 30

Q. What “component of your result object details cluster assignment/membership?”

```
km$cluster
```

[illegible]

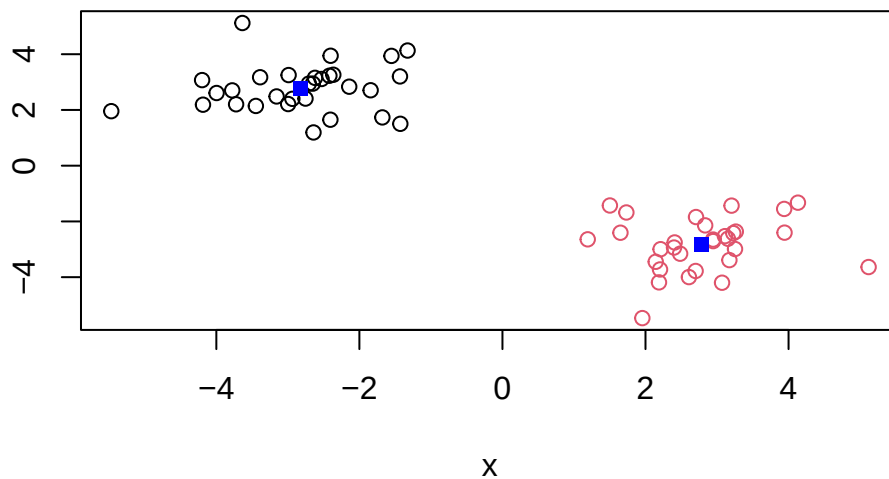
Q. What component of your result object results in cluster centers

```
km$centers
```

```
      x  
1 -2.826674  2.779687  
2  2.779687 -2.826674
```

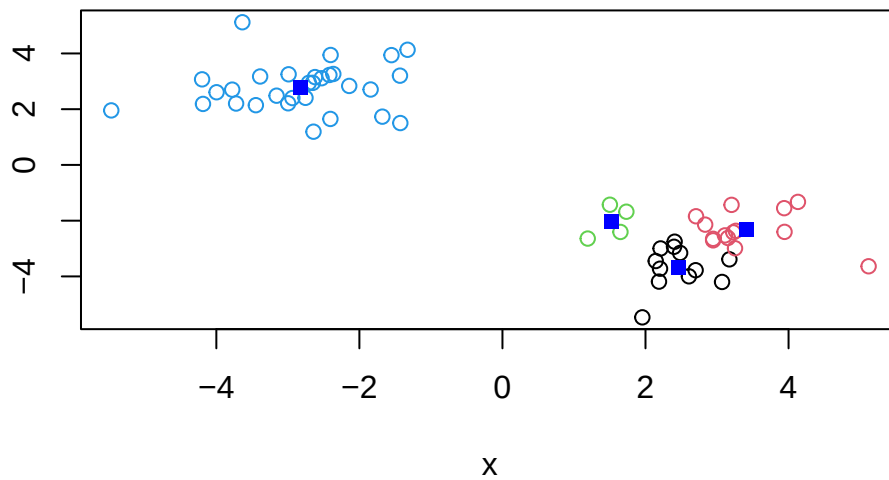
Q. Plot `z` colored by the `kmeans` cluster assignment and add cluster centers as blue points

```
plot(z, col=km$cluster)  
points(km$centers, col="blue", pch=15)
```



Run a K-means clustering and plot the results asking for 4 clusters

```
km4 <- kmeans(z, 4)  
plot(z, col=km4$cluster)  
points(km4$centers, col="blue", pch=15)
```



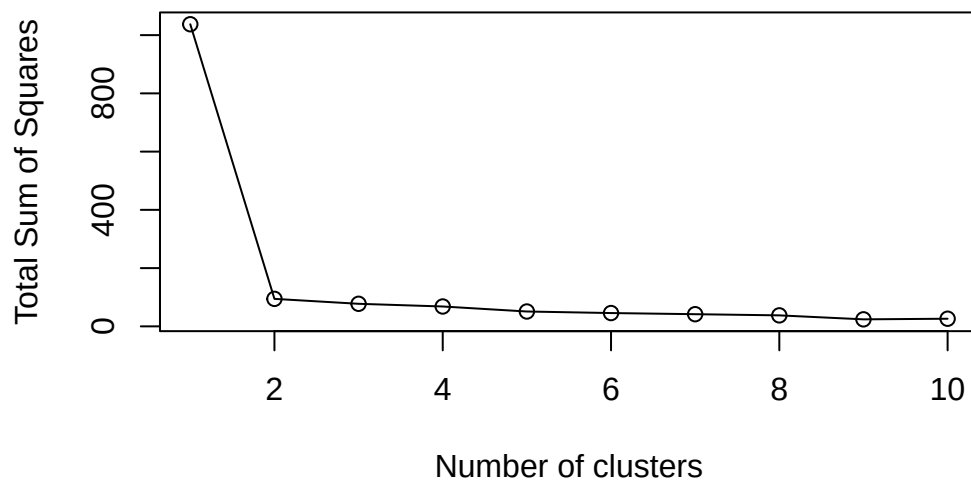
N.B You need to tell K-means the number of clusters

One approach is to try different values for `centers` and then pick the best

```
ans <- NULL
for (i in 1:10){
  km <- kmeans(z, centers=i)
  ans <- c(ans, km$tot.withinss)
}
ans
```

```
[1] 1037.36200  94.42347  77.43591  68.18060  50.95750  45.66389
[7]  41.78733  37.71462  24.04493  26.15680
```

```
plot(ans, typ="o", xlab="Number of clusters", ylab="Total Sum of Squares")
```



Hierarchical Clustering

The main function in “base” R for Hierarchical Clustering is called `hclust()`

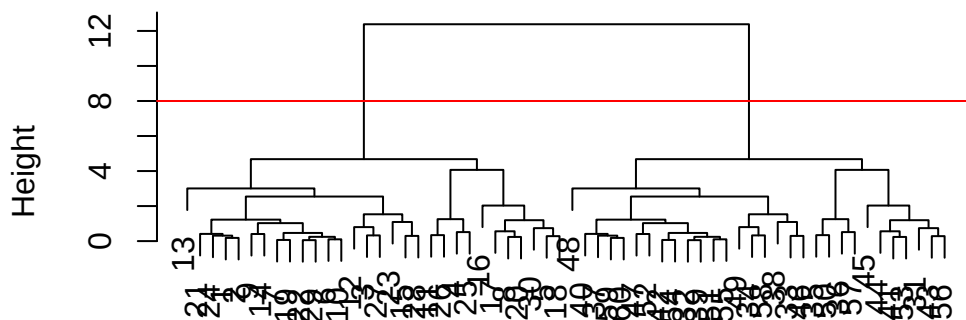
This function does not take your “raw” data for clustering. You must first build a “distance matrix” from your data and pass this as input to `hclust()`.

```
d <- dist(z)
hc <- hclust(d)
```

There is a bespoke `plot()` method for `hclust()` result objects.

```
plot(hc)
abline(h=8, col="red")
```

Cluster Dendrogram



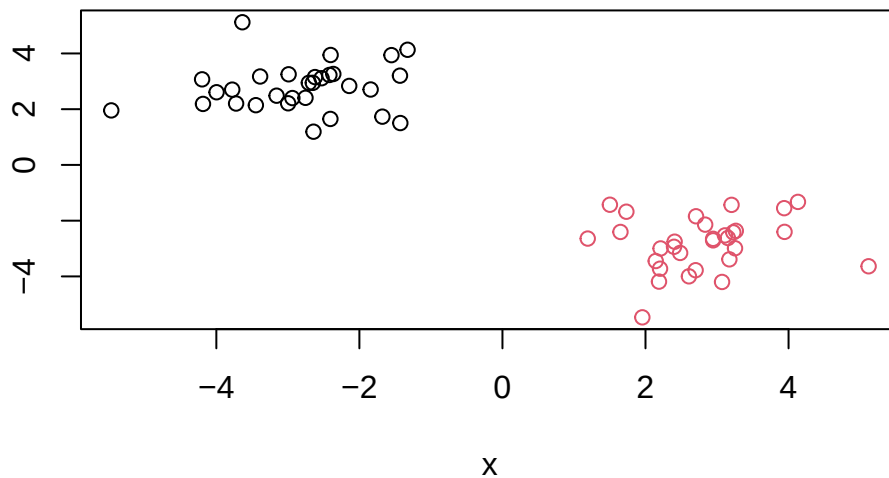
d
hclust (*, "complete")

Once we have our `hclust` object (our “tree” of “cluster dendrogram”) we can “*cut*” the tree to reveal the clustering pattern.

```
tree <- cutree(hc, h=8)
```

Q. Make a plot of `z` with your `hclust` results (i.e colored by cluster membership)

```
plot(z, col=tree)
```

Hands on with Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets.

Analysis of UK food data

Let's look at some data on the eating habits of folks from the UK to see if there are patterns and trends that have some regions being distinct from others.

Data Import

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
x
```

		X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66	
2	Carcass_meat	245	227	242	267	

3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139
7	Fresh_potatoes	720	874	566	1033
8	Fresh_Veg	253	265	171	143
9	Other_Veg	488	570	418	355
10	Processed_potatoes	198	203	220	187
11	Processed_Veg	360	365	337	334
12	Fresh_fruit	1102	1137	957	674
13	Cereals	1472	1582	1462	1494
14	Beverages	57	73	53	47
15	Soft_drinks	1374	1256	1572	1506
16	Alcoholic_drinks	375	475	458	135
17	Confectionery	54	64	62	41

The data is made available in CSV format so we can use the `read.csv()` function for import to R.

Tidy data

**Q1. How many rows and columns are in your new data frame named `x`?
What R functions could you use to answer this question?**

You can use `dim()` to input `x` and return the row and column numbers. There are 17 rows and 5 columns as shown below.

```
dim(x)
```

```
[1] 17  5
```

Fix anything that went wrong with data import.

```
rownames(x) <- x[,1]
x <- x[,-1]

x
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267

Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187
Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494
Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506
Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

This can also be solved by calling `row.names` during the initial csv read

```
x <- read.csv(url, row.names=1)
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

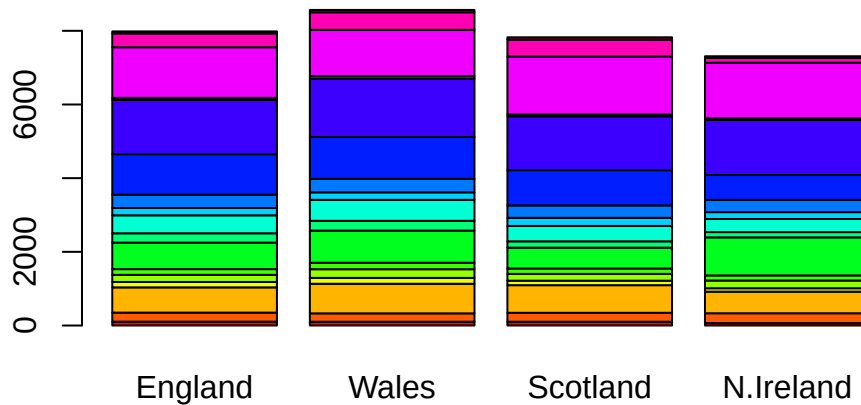
Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer the second approach as it’s much simpler, and you can rerun it without having to reset the value of `x`, however I assume that the first approach would be more robust for removing data for example, if the goal is to navigate to a certain place in the dataframe. It’s mainly weighed down by its inability to repeat.

Exploratory analysis

Make some plots to help make sense of obvious trends...

```
barplot(as.matrix(x), col=rainbow(nrow(x)))
```

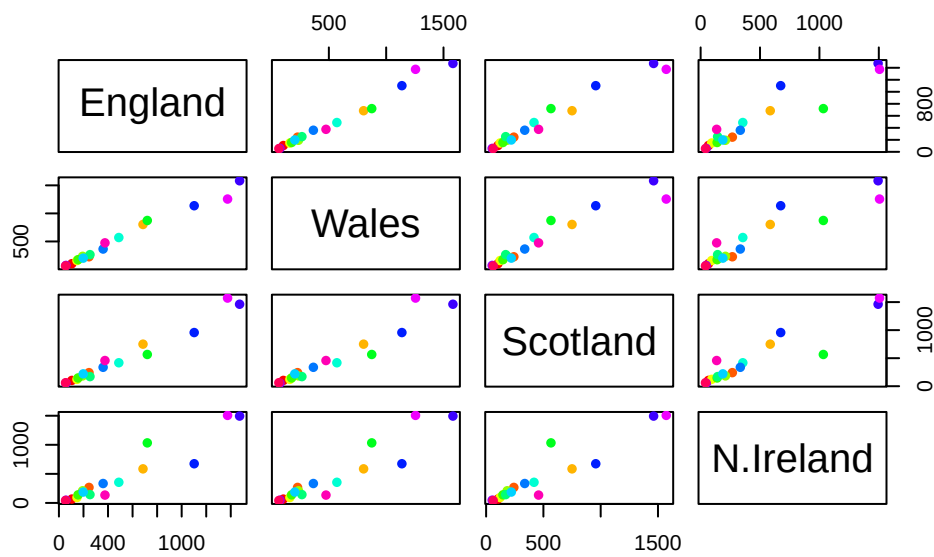


Q3: Changing what optional argument in the above `barplot()` function results in the following plot?

Removing `beside=T` changes it into the above plot.

Q5: We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

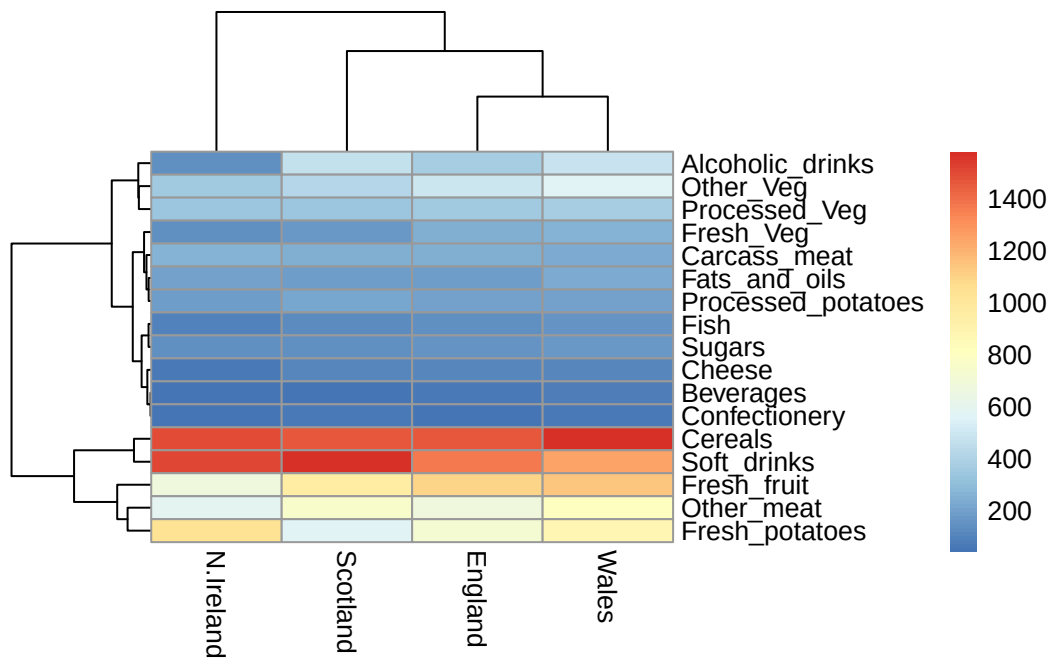
```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



If it's on the diagonal, both countries consume similar amounts of each of the foods. Off diagonal means different consumption in some way.

```
library(pheatmap)

pheatmap( as.matrix(x) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

It seems that for the big two groups, Scotland England and Wales cluster together and N. Ireland seems to be in a separate cluster. England and Wales seem to cluster together compared to Scotland as well. It seems difficult to tell the main differences since while there seem to be differences and general trends, it remains fully unclear

##PCA

The main function in “base” R for PCA is called `prcomp()`. This function wants the observations to be rows and the variables to be columns.

So here we need to take the transpose of our `x` input object

```
pca <- prcomp(t(x))
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	3.176e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

The returned `pca` object has components that we can use to make our main result figures:

```
pca$x
```

	PC1	PC2	PC3	PC4
England	-144.99315	-2.532999	105.768945	-4.894696e-14
Wales	-240.52915	-224.646925	-56.475555	5.700024e-13
Scotland	-91.86934	286.081786	-44.415495	-7.460785e-13
N.Ireland	477.39164	-58.901862	-4.877895	2.321303e-13

The main result from this analysis is called a “PC score plot” or “ordination plot” or “PC plot”

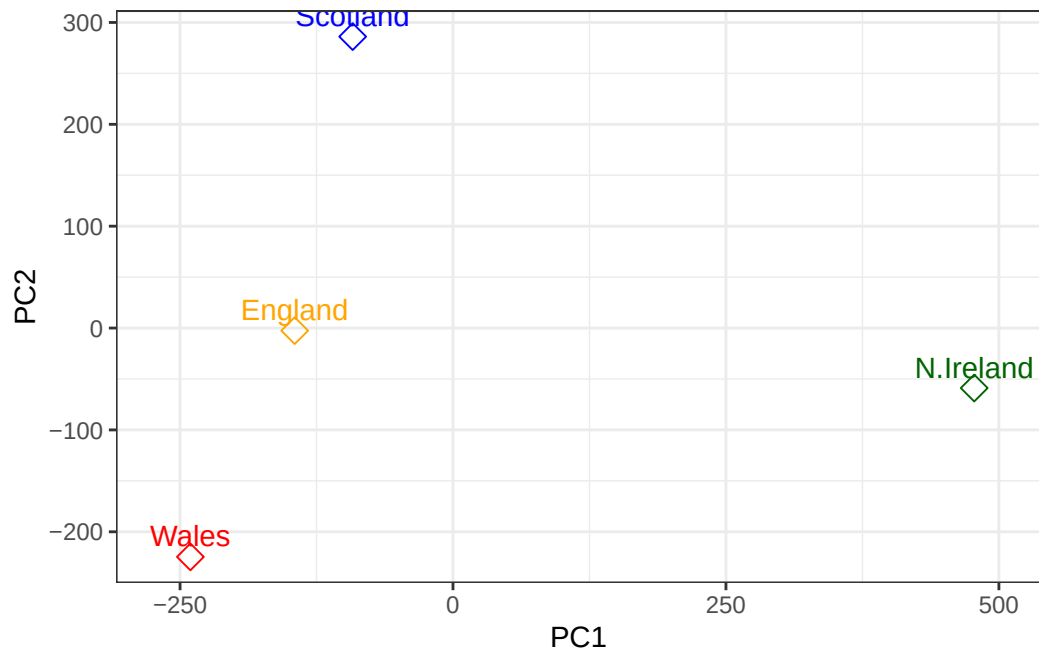
Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

I combined the two questions’ results below:

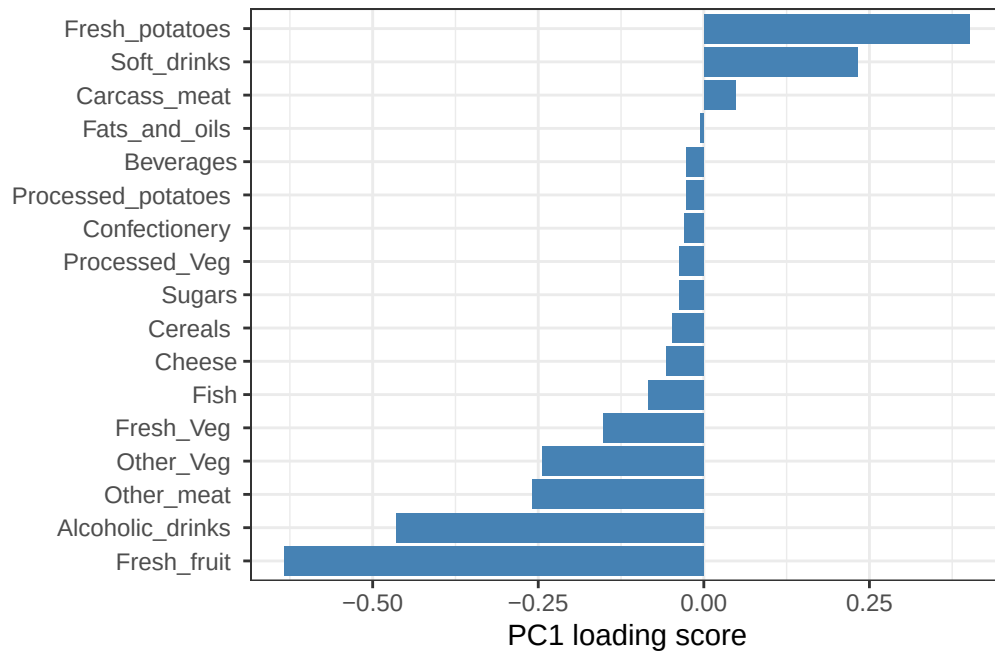
```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)
mycols <- c("orange", "red", "blue", "darkgreen")

# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3, color=mycols, shape=5) +
  geom_text(vjust = -0.5, col=mycols) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



This plot shows the difference in terms of PC1 and PC2 between regions in the UK, and shows clear statistical trends that North Ireland is exceptionally different compared to the other 3 regions.

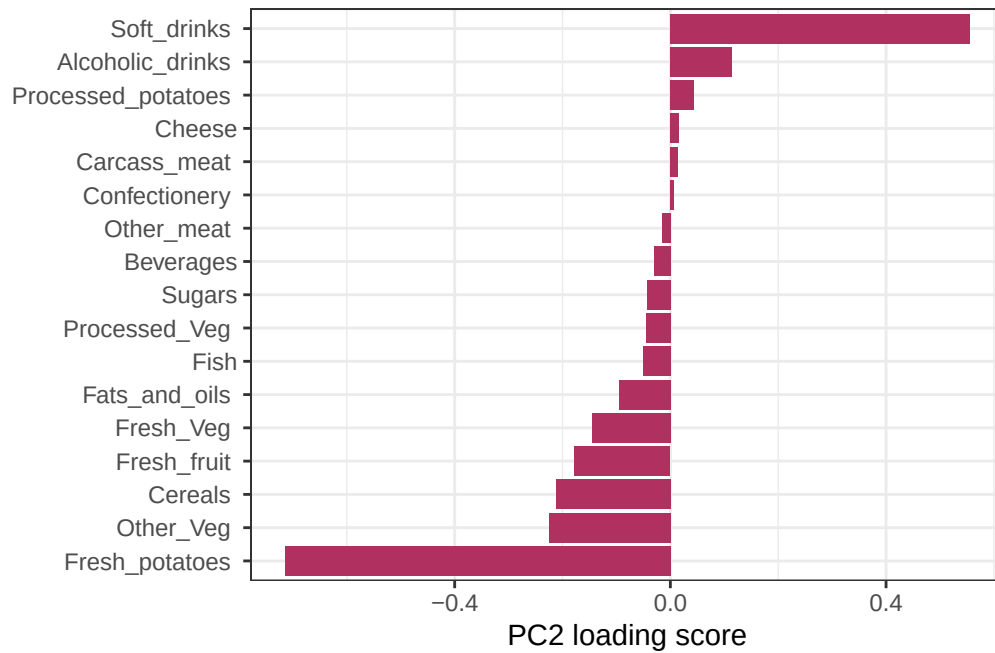
```
ggplot(pca$rotation) +  
  aes(PC1, reorder(rownames(pca$rotation), PC1)) +  
  geom_col(fill= "steelblue") +  
  xlab("PC1 loading score") +  
  ylab("") +  
  theme_bw()
```

This output plot is basically what variables affect PC1 the most in positive and negative ways, and determine the importance of each variable on PC1.

Q9: Generate a similar ‘loadings plot’ for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

```
ggplot(pca$rotation) +
  aes(PC2, reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill= "maroon") +
  xlab("PC2 loading score") +
  ylab("") +
  theme_bw()
```



PC2 mainly tells us about soft drinks and fresh potatoes, with the former positively influencing the PC2 and the latter negatively influencing it.